

Storico Domande Esami SO (2017-2023)

Autore: Paolo Masciullo (Aggiornate da: Nicola Cassano e Febbraro Samuele)

Linux

1. Si scriva il comando complesso che permetta di contare quanti processi all'interno della macchina host hanno un PID caratterizzato da un numero pari.

```
ps | cut -f1 -d' ' | grep -Ec '^[[:digit:]]*[02468]$'
```

2. Si scriva un unico comando (pipeline) per fornire la lista dei file semplici all'interno della home directory per cui gli utenti 'altri' del sistema possengano esclusivamente il permesso di lettura e il cui nome sia una stringa che termina con una vocale seguita da una cifra.

```
ls -l | grep -E '^-.{6}r--.*[aeiou][[:digit:]]$'
```

3. Spiegare l'utilità del comando kill sottolineando le differenze nell'utilizzo con i due flag elencati qui sotto:

- Kill -15 PID
- Kill -9 PID

Il comando kill permette di inviare un segnale al processo specificato come argomento attraverso il suo PID. Con il flag -15 (default) si invia al processo specificato il segnale SIGTERM che indica al processo di terminare, il processo che riceve questo segnale lo può elaborare e decidere di terminare in un punto sicuro della sua elaborazione. Con il flag -9 si invia al processo specificato il segnale SIGKILL, questo segnale fa terminare immediatamente il processo senza che esso possa elaborare questo segnale

4. Scrivere una sequenza di comandi che consenta di ordinare in ordine alfabetico il contenuto di un file di nome prova.testo e di estrarre dal file ordinato le prime 5 righe scrivendole in appendice sul file output.txt presente nella home directory dell'utente loggato.

```
cat prova.testo | sort | head -5 >> output.txt
```

5. Si spieghino le differenze tra i file etc/shadow e etc/passwd. Si indichi come sono strutturati, qual è la rispettiva funzione e chi può accedervi.

Il file /etc/passwd è un file accessibile a tutti gli utenti e contiene informazioni circa gli utenti del sistema, mentre il file /etc/shadow è un file accessibile solo dal superutente e contiene informazioni circa le password degli utenti del sistema.

Il file /etc/passwd è così strutturato: user:password:userID:groupID:GECOS:homedir:shell

- User, indica il nome dell'utente
- Password, indica la password cifrata dell'utente, se assume uno tra i caratteri * ! !! vuol dire che la password è presente in forma criptata nel file /etc/shadow
- userID, indica l'ID dell'utente
- groupID, indica l'ID del gruppo principale a cui appartiene l'utente
- GECOS, indica informazioni aggiuntive sull'utente
- Homedir indica la home directory dell'utente
- Shell, indica la shell utilizzata dall'utente

Il file /etc/shadow è così strutturato: user:password:lastchange:min:max:warn:inactive:expire

- User, indica il nome dell'utente
- Password, indica la password criptata(13 caratteri)
- Lastchange, indica il numero di giorni trascorsi dal 1 gennaio 1970 all'ultima modifica della password
- Min, indica il numero minimo di giorni che devono trascorrere dall'ultima modifica prima che la password sia nuovamente modificabile
- Max, indica la durata massima in giorni della password
- Warm, indica il numero di giorni di preavviso all'utente prima che la password venga invalidata
- Inactive, indica il numero di giorni di inattività della password prima che l'account venga disabilitato
- Expire, numero di giorni trascorsi dal 1 gennaio 1970 trascorsi i quali il login non può essere più effettuato

6. Si scriva un unico comando (pipeline) in grado di restituire la lista delle directory all'interno della home per la quale il gruppo proprietario ha (esclusivamente) il permesso di lettura e il cui nome è una stringa che termina con tre cifre.

```
ls -l ~/ | grep -E 'd.{3}r-{2}.*[[:digit:]]{3}$'
```

7. Chiarire le differenze esistenti tra i due comandi seguenti:

- Chmod g+w nome.prova
- Chmod 664 nome.prova

Posto che la ACL del file nome.prova sia rw-r-----. Si dica anche quali categorie di utenti possono lanciare il comando. Le differenze tra i due comandi è innanzitutto il formato con cui vengono rappresentate le ACL, nel primo caso l'ACL è rappresentata in forma letterale e il comando in questo caso aggiunge il permesso di scrittura al gruppo proprietario del file; nel secondo caso il formato dell'ACL è ottale, in cui ognuna delle tre cifre rappresenta i permessi codificati in ottale forniti rispettivamente all'utente proprietario, al gruppo proprietario e agli altri utenti, in questo caso il comando fornisce all'utente proprietario i permessi di lettura e scrittura (6), al gruppo proprietario vengono forniti i permessi di lettura e scrittura (6), e agli altri utenti viene fornito solo il permesso di lettura.

Indipendentemente dall'ACL specificata in questo caso per il file nome.prova solo l'utente proprietario del file o il superuser può lanciare il comando chmod.

8. Il file risultati_esame.txt contiene nome, cognome e voto conseguito per ciascuno studente:

- mario rossi 22
- paolo rossi 19
- rocco verdi 24
- marco rossi 23
- sergio bianchi 26
- mario rossi 22
- rosa barbieri 28

Si scriva il comando per mostrare sullo standard output l'elenco degli studenti che hanno una votazione maggiore o uguale a 22, con ordinamento numerico decrescente eliminando le righe ripetute.

```
grep -E '2[2-9]|30' risultati_esame.txt | sort -nr | uniq
```

9. Contare quanti processi all'interno della macchina host non appartengono all'utente identificato dal nome UserA.

```
ps -f | grep -cv '^UserA'
```

10. Elaborando il contenuto del file `/etc/passwd` si scriva il comando per mostrare sullo standard output l'elenco di tutti gli utenti del sistema diversi da root che utilizzano bash come shell di default.

```
cat /etc/passwd | grep -vE '^root' | grep -E 'bash$'
```

11. Scrivere il comando Linux che seleziona dal file `/etc/passwd` e stampa a video tutte le righe che contengono due cifre dispari, separate da un qualsiasi carattere.

```
cat /etc/passwd | grep -E '[13579].[13579]'
```

12. Si scriva il comando Linux che permette di aggiungere un account utente al sistema con le seguenti caratteristiche:

- Nome: mario
- Home directory: /home/mario
- Shell: /bin/bash
- Gruppo principale: mario
- Altri gruppi: users, mail, printer

```
sudo useradd -m -d /home/mario -s /bin/bash -g mario -G users,mail,printer mario
```

13. Spiegare il significato delle seguenti variabili di ambiente e chiarire preliminarmente cosa permette di fare questo strumento in Linux e in altri sistemi programmati: (PATH, HOME, USER, SHELL, PWD).

Le variabili di ambiente sono dei nomi contenenti dei valori modificabili a cui i processi facenti parte di quel ambiente hanno accesso, sono particolarmente utili per identificare percorsi di directory, dati di configurazione ecc...

- PATH: contiene il percorso di ricerca per i file eseguibili
- HOME: contiene il percorso della directory home dell'utente loggato
- USER: contiene il nome dell'utente loggato
- SHELL: contiene il percorso dell'eseguibile della shell di default
- PWD: contiene il percorso della directory corrente

14. Scrivere una pipeline di comandi che fornisca in output il numero di file semplici (inclusendo anche quelli nascosti), presenti all'interno della directory corrente, per i quali:

- L'utente proprietario ha almeno i permessi di lettura e scrittura;
- Il nome dell'utente proprietario inizia con la lettera 'a';
- Il gruppo proprietario ha solo il permesso di lettura ed il suo nome termina con 'so'.

```
ls -la | grep -E '^-rw.r--.*\<a.*so\>'
```

15. Utilizzando esclusivamente i comandi ls e sed si visualizzi l'elenco in formato esteso dei file presenti all'interno della directory corrente, eliminando la colonna relativa all'ACL.

```
ls -l | sed -e 's/^.{11}\s//'
```

16. Scrivere una pipeline di comandi che fornisca in output il numero di file semplici(anche nascosti), presenti all'interno della directory corrente, di cui l'utente proprietario possiede almeno i permessi di scrittura.

```
ls -la | grep -Ec '^-.w'
```

17. Spiegare il comando `cat lista_nomi.txt | head -4 | grep '\(3\|[34]\)$' -v > output.txt` e indicare l'effetto che esso produce se il file `lista_nomi.txt` è così fatto:

- mario rossi 080 541234
- paolo rossi 083 434232
- rocco verdi 093 3424033
- marco rossi 066 091312
- sergio bianchi 083 311234
- fabio giallo 080 123080
- rosa barbieri 081 312313

Il comando cat cattura il contenuto del file lista_nomi.txt attraverso la pipeline con il comando head -4 visualizzo solo le prime 4 righe, con un ulteriore pipeline con il comando grep 'regex' matcho le righe che non hanno 3 o 3-4 alla fine della riga, il risultato è ridirezionato in modalità truncate nel file output.txt il cui contenuto sarà:

paolo rossi 083 434232

marco rossi 066 091312

18. Spiegare qual è l'effetto del seguente comando: `$ls -aF1 | grep '^\. '`

Con il comando ls stampo sullo standard output tutti i file e directory presenti nella directory corrente, con il flag -a visualizzo anche i file nascosti, con il flag -F viene aggiunto un simbolo per ogni file che ne identifica il tipo, con il flag -1 visualizzo l'intero output su una sola colonna, con il comando di pipe invio tutto l'output in input al comando grep la cui regex matcha le righe che contengono all'inizio della riga un punto

19. Si scriva il comando per inviare l'output del list della directory /etc/var/www/html sul file result.txt presente nella home directory dell'utente loggato filtrando tutte e solo le directory di cui il gruppo proprietario ha il permesso di lettura ed esecuzione.

`ls -l /etc/var/www/html | grep -E 'd.{3}r-x' > ~/result.txt`

20. Sia assegnato il seguente albero di directory e si supponga di avere i privilegi di superutente:

```

/
├── var/
│   └── www/
│       └── html/
├── etc/
│   ├── passwd
│   ├── default/
│   └── shadow

```

Dire come l'albero risulta modificato dopo la seguente sequenza di comandi:

`cd /etc/default` mi posiziono all'interno della directory default

`cp ../passwd ./` copio il file passwd nella directory default

`cd /var/www` mi posiziono all'interno della directory www

`mv ./html /` sposto la directory html nella directory root (/)

`touch ./html` creo un nuovo file html in /var/www

mkdir prova creo una nuova directory prova in /var/www

cd – torno nella directory precedentemente usata ossia in /etc/default

touch ./file creo un file di nome file nella directory /etc/default

/

html/

var/

www/

html

prova/

etc/

passwd

default/

passwd

file

shadow

21. Spiegare il comando `cat lista_nomi.txt | head -3 | grep '\<r.*i\>.*$' -v > output.txt` e indicare l'effetto che esso produce se il file `lista_nomi.txt` è così fatto:

- mario rossi 080 541234
- paolo rossi 081 434234
- luigi verdi 094 3424080
- marco rossi 066 091312
- sergio bianchi 080 311231
- fabio giallo 080 123080
- rosa barbieri 081 312313

Il comando `cat` cattura il contenuto del file `lista_nomi.txt` attraverso la pipeline con il comando `head -3` visualizzo solo le prime 3 righe, con un ulteriore pipeline con il comando `grep 'regex'` matcho le righe che non hanno parole che iniziano con 'r' e finiscono con 'i', il risultato è ridirezionato in modalità troncata nel file `output.txt` il cui contenuto sarà:

luigi verdi 094 3424080

22. Spiegare, motivando la risposta, quali sono gli effetti del comando `[user@PC]$chmod 744 ./file_A` relativamente al file `-rwxrwxr-x 2 user group_a 55231 Apr 23 20:32 file_A`.

Il comando `chmod` può essere eseguito poiché il proprietario del file è l'utente che ha chiamato il comando. L'effetto è quello di impostare i permessi ad utente gruppo e altri, il risultato finale è:

`-rwxr--r-- 2 user group_a 55231 Apr 23 20:32 file_A`

23. Spiegare il significato dei campi del seguente output del comando:

`$ls -la: drwxrwxr-x 2 pippo group_mail 52874 Feb 8 16:36 .file`

La struttura dell'output è:

- Primo carattere rappresenta il tipo di file: d = directory
- rwxrwxr-x rappresentano l'ACL rispettivamente rwx per l'utente proprietario, rwx per il gruppo, r-x per gli altri utenti
- 2 rappresenta il numero di hard link al file
- Pippo è il nome dell'utente proprietario
- Group_mail rappresenta il gruppo di appartenenza dell'utente proprietario
- 52874 rappresenta l'inode del file
- Feb 8 16:36 rappresenta il timestamp di ultima modifica
- .file è il nome del file, il punto indica che è un file nascosto mandato in output grazie al flag -a

24. Si scriva il comando per inviare l'output del list della directory /home/utenti sul file lista.txt presente nella home dell'utente attualmente loggato filtrando solo i file standard su cui gli utenti proprietari hanno almeno i permessi di lettura e scrittura.

```
ls -l /home/utenti | grep -E '^-rw.' > ~/lista.txt
```

25. Si scriva il comando per inviare l'output del list della directory /etc/var/www/html sul file output.txt presente nella home directory dell'utente loggato filtrando tutti e soli gli elementi che iniziano con la lettera 'm' o 'M'.

```
cd; ls /etc/var/www/html | grep -E '[mM]' > output.txt
```

26. Sia assegnato il seguente albero di directory e si supponga di avere i privilegi di superutente:

```
/
├── var/
│   └── www/
│       └── html/
├── etc/
│   ├── passwd
│   ├── default/
│   └── shadow
```

Dire come l'albero risulta modificato dopo la seguente sequenza di comandi:

```
cd /etc/default mi posiziono all'interno della directory default/
```

```
cp ../pa* ./ copio il file password, presente nella directory padre di default/ ossia etc/, nella directory default/
```

```
cd /var/www mi posiziono nella directory www/
```

```
mv ./html / sposto la directory html/ nella directory root /
```

```
touch ./html creo un file di nome html nella directory www/
```

```
mkdir prova creo la directory prova/ nella directory www/
```

l'albero delle directory risultante è:

/

html/

var/

www/

html

prova/

etc/

passwd

default/

passwd

shadow

27. Spiegare l'utilità del comando kill sottolineando le differenze nell'utilizzo con i due flag elencati qui sotto:

- Kill -15 PID

- Kill -9 PID

28. Si dica con precisione quali informazioni sono contenute all'interno del file /etc/passwd e se questo è accessibile da un utente senza privilegi di amministratore.

Il file /etc/passwd è così strutturato

User:passwd:userID:groupID:GECOS:homedir:shell

- User: nome dell'utente
- Passwd: password criptata o 'x' se si utilizza il file shadow
- userID: id dell'utente
- groupID: id del gruppo primario di appartenenza dell'utente
- GECOS: informazioni addizionali sull'utente
- Homedir: directory home dell'utente
- Shell: shell utilizzata di default

29. Utilizzare i comandi sed e cut per costruire il file lista_utente.testo, nella home directory dell'utente attualmente loggato, in modo che contenga l'elenco di tutti i nomi degli utenti del sistema insieme alle relative home dir separati dal carattere '-' (esempio: user-/home/user).

cut -f 1,6 -d : /etc/passwd | sed -e 's:/-/ /' > ~/lista_utente.testo

30. Si scriva il comando Linux che permette di aggiungere un account utente al sistema con le seguenti caratteristiche:

- Nome: mario
- Home directory: /home/mario
- Shell: /bin/bash
- Gruppo principale: mario
- Altri gruppi: users, mail, printer

```
sudo useradd -m -d /home/mario -s /bin/bash -g mario -G users, mail, printer mario
```

31. Come è possibile sapere quali utenti (esclusivamente il nome), il cui account cominci per 'm' o 'M', sono titolari di un processo denominato gnome-terminal su terminali diversi? Eliminare eventuali righe duplicate.

```
ps ax -f | grep '^[mM].*gnome-terminal$' | cut -f 1 -d ' ' | uniq
```

 Da rivedere

32. Scrivere il comando linux per visualizzare le informazioni estese di tutti i file semplici presenti all'interno della directory corrente, per i quali l'utente proprietario ed i membri del gruppo proprietario hanno almeno i privilegi di lettura e scrittura e tutti gli altri utenti possiedono solo il permesso di lettura. Le informazioni estratte devono essere salvate su un file di nome output.txt.

```
ls -l | grep -E '^(rw.){2}r-{2}' > output.txt
```

33. Il file elenco.txt ha il seguente contenuto:

- Rossi Mario
- Rossi Paolo
- Verdi Giuseppe
- Bollo Franco
- Bianchi Alessandra
- Cerri Elena
- Dodi Francesco

Si costruisca il file risultato.txt che contenga in ordine alfabetico tutti gli elementi con cognome che inizi per R o per B.

```
cat elenco.txt | grep -E '^[RB]' | sort > risultato.txt
```

34. Spiegare l'effetto del comando seguente: `grep -E '^.*\<1[2-9]\.{6}.*$' ./prova/*`

Il comando seguente matcha tutti i file e directory all'interno della directory prova che abbiano:

(`^.*`) zero o più occorrenze di un carattere qualsiasi ad inizio riga, un 1 seguito da (`\<[2-9]`) un numero compreso tra 2 e 9 ad inizio parola, seguito da (`\.{6}`) 6 punti seguito da (`.*$`) zero o più occorrenze di un carattere qualsiasi a fine riga.

35. La directory corrente contiene il file binary.txt le cui righe sono costituite da stringhe formate solo da combinazioni di '1' e '0'. Si scriva il comando UNIX per poter estrarre da questo file solo le righe che contengono almeno tre '0' consecutivi e produrre il risultato nel file di nome output.txt presente nella home directory dell'utente attualmente loggato al sistema.

```
grep -E '0{3}' ./binary.txt > ~/output.txt
```

36. Spiegare il significato dei campi del seguente output del comando `$ls -la`:

```
drwxrwxr-x 2 user_group 34234 Feb 9 15.34 .file_name
```

`ls -la` manda in output i file e le directory in forma estesa (`-l`), compreso i file nascosti (`-a`) receduti da un `'.'`

- Il primo carattere indica il tipo di file, in questo caso una directory (`d`)
- I successivi 9 caratteri divisi in 3 gruppi di 3 indicano l'ACL rispettivamente di utente proprietario, gruppo proprietario e altri utenti, in questo caso `rw` all'utente proprietario e al gruppo proprietario (lettura, scrittura, esecuzione), `r-x` agli altri utenti (lettura ed esecuzione)
- 2 indica il numero di hard link

- User_group indica l'utente proprietario e il gruppo proprietario
- 34234 indica la dimensione in byte del file o directory
- Feb 9 15.34 indica il time stamp di ultima modifica
- .file_name nome del file, il '.' Indica che è un file nascosto

37. Si scriva il comando per inviare l'output del list della directory /var/www/html sul file web.list presente nella home directory dell'utente loggato e attivo filtrando tutti e soli i file o le directory nascoste.

```
ls -la /var/www/html | grep -E '^\. ' > /home/nome_utente/web.list
```

38. Si scriva il comando per inviare l'output del list della directory /home/utenti sul file lista.txt presente nella home dell'utente attualmente loggato filtrando solo i file standard su cui gli utenti proprietari hanno almeno i permessi di lettura e scrittura.

```
ls -l /home/utenti | grep -E '^-rw ' > /home/nome_utente/lista.txt
```

39. Scrivere il comando UNIX per visualizzare, dal più piccolo al più grande, i numeri contenuti all'interno del file numeri presente nella directory corrente e produrre il risultato nel file di nome fileA nella home directory dell'utente attualmente loggato al sistema

```
sort -n ./numeri > /home/nome_utente/fileA
```

40. Spiegare, motivando la risposta, quali sono gli effetti del seguente comando: [user@PC] \$chmod 744 ./file_A Relativamente al file: -rwxrwxr-x 2 user group_A 55231 Apr 23 20:32 file_A

Il comando chmod può essere eseguito poiché il proprietario del file è l'utente che ha chiamato il comando. L'effetto è quello di impostare i permessi ad utente gruppo e altri, il risultato finale è:

```
-rwxr--r-- 2 user group_a 55231 Apr 23 20:32 file_A
```

41. Si supponga che la CWD sia ~/dir. Descrivere la modalità per copiare il file ~/dir/file.prova nella home directory, evitando di sovrascrivere un eventuale file esistente.

```
cp -n file.prova ..
```

42. Indicare quali sono i possibili comandi linux che possono essere adoperati per la cancellazione di una directory.

rm -r cancella ricorsivamente una directory e i contenuti al suo interno; rmdir rimuove una directory SOLO se vuota. Col flag -ignore-fail-on-non-empty non torna l'errore relativo alla directory "piena" e quindi non cancellabile.

43. Disegnare l'albero di directory e file generato dalla seguente sequenza di comandi digitati avendo i permessi di amministratore di sistema. La directory di partenza è la home directory dell'utente attualmente loggato nel sistema che si chiama user.

```
root@ubuntu:/home/user# mkdir dir1 dir2
```

```
root@ubuntu:/home/user# cd dir1
```

```
root@ubuntu:/home/user# touch file1.txt
```

```
root@ubuntu:/home/user# mv file1.txt ..
```

```
root@ubuntu:/home/user# cd ../dir2
```

```
root@ubuntu:/home/user# touch file2.txt
```

```
root@ubuntu:/home/user# cp ../file1.txt .
```

```
/
```

```
home/
```

```
user/
```

```
dir1/
```

```
dir2/
```

```
file1.txt
```

```
file2.txt
```

```
file1.txt
```

44. Scrivere il comando UNIX per visualizzare in ordine alfabetico i file contenuti nella directory corrente e produrre il risultato nel file di nome fileA nella directory dell'utente attualmente loggato al sistema.

```
ls | sort > /home/nome_utente/fileA
```

45. Si indichi la struttura tipica dell'output del comando ps aux. Spiegare il significato delle flag e indicare un possibile comando alternativo. Il comando ps permette di visualizzare una schermata statica dei processi attualmente attivi sulla shell.

Il comando ps aux permette l'aggiunta di tre funzioni: "a" visualizza tutti i processi attivi, indipendentemente dall'utente che l'abbia avviato; "u" permette una visualizzazione user oriented; "x" comprende anche i processi senza terminale. Un comando alternativo è top, il quale fa visualizzare una schermata in continuo aggiornamento e non statica.

46. Spiegare a cosa serve il comando top chiarendo le differenze principali con comandi analoghi. Specificare in dettaglio quali informazioni esso produce in output.

Il comando "top" mostra una vista in tempo reale dei processi attivi. Esso mostra i processi che più incidono sull'utilizzo della CPU. Di default in cima vengono mostrati i processi che usano in percentuale più CPU. Il comando "ps" invece mostra i processi correntemente attivi sul sistema e di default mostra solo quelli dell'utente. Per mostrare anche i processi non appartenenti all'utente con o senza terminale in formato user-oriented si utilizza il flag -aux.

47. Si indichi la struttura tipica dell'output del comando man.

Il comando man formatta e mostra pagine di guida in linea. La sintassi è: man -[fkW] [-S sezione] [sezione] [-P paginatore] La sua struttura è formata da:

- NAME: nome comando e breve descrizione.
- SYNOPSIS: sintassi del comando.
- DESCRIPTION: descrizione dettagliata.
- OPTIONS: lista di tutti i flag con relativa descrizione.

48. Si chiarisca la differenza di utilizzo tra i comandi help, man e h riportandone la sintassi. Chiarire la differenza tra i tre diversi tipi di comandi che una shell può eseguire.

-Interni (built-in): funzioni elementari eseguite dalla shell. La loro esecuzione non provoca la creazione di un nuovo task poiché il codice fa parte dell'eseguibile stesso della shell.

-Esterni: file contenenti programmi in formato eseguibile. La loro esecuzione provoca la creazione di un nuovo task figlio della shell da cui sono avviati.

-Script: file testuali in linguaggio shell contenenti sequenze di istruzioni e comandi interpretabili dalla shell. La loro esecuzione provoca la creazione di una nuova istanza della shell, figlia della shell stessa.

49. Qual è la differenza essenziale tra i comandi jobs, top e ps?

- JOBS: mostra lo stato dei jobs attivi nella sessione di terminale corrente.
- TOP: time of process, mostra in modo interattivo lo stato dei processi in tempo reale.
- PS: process status, mostra lo stato dei processi ma non interattivamente.

50. Si scriva la sinossi del comando UNIX per cambiare il gruppo di un file specificando i prerequisiti indispensabili.

- chmod [ugoa] [+|=] [rwx] modifica i permessi relativi all'utente e gruppo proprietario, e per gli altri.
- chown [utente.gruppo] <percorso> cambia proprietario.
- chgrp [gruppo] <percorso> cambia il gruppo proprietario.

Tutti i comandi possono essere utilizzati solo dal proprietario del file in esame o il superutente

Android

1. Spiegare in dettaglio il meccanismo di sicurezza basato sui permessi del SO Android.

In android ogni app è confinata nella sua sandbox, questo implica che per accedere a dati e servizi di sistema o di altre app bisogna fare richiesta esplicita dei permessi. Questi permessi vanno definiti all'interno del file AndroidManifest.xml.

Un permesso può essere:

- Custom: propri di un'applicazione e definiti nel android manifest mediante il tag <permission>
- Predefinito, consente l'accesso alle funzionalità di sistema

2. Descrivere le tipologie di intent previsto dal SO Android.

Un intent è un'azione tra più componenti di un app.

Le tipologie di intent sono:

- Intent esplicito, azione che coinvolge un destinatario univocamente identificato (es. aprire la sezione impostazioni della propria app)
- Intent implicito, azione generica in cui non è specificato il destinatario (es. aprire la pagina web di google, ma con quale browser?)

3. Elencare brevemente le fasi del ciclo di vita di una Activity Android.

Le fasi di un activity nel suo ciclo di vita sono:

- Attiva, in primo piano
- In pausa, oscurata parzialmente
- Terminata, chiusa e non più visibile

I metodi per gestire queste 3 fasi sono: onCreate(), onStart(), onResume(), onPause(), onStop(), onDestroy()

4. Descrivere i meccanismi per il risparmio energetico adottati dal SO Android. Si facciano degli esempi.

Android, per il risparmio energetico adotta un meccanismo basato sui wakelocks, ossia un'app per mantenere attiva una risorsa deve fare richiesta esplicita e ottenere un wakelock, in assenza del quale android disattiva la risorsa per risparmiare energia. Un esempio può essere l'utilizzo dello schermo, un app, per mantenere attivo lo schermo del dispositivo deve possedere il wakelock opportuno in assenza del quale lo schermo viene spento.

5. Spiegare il funzionamento del Binder IPC driver ed il suo ruolo nella gestione della sicurezza nel Sistema Operativo Android.

Il Binder IPC Driver stabilisce un canale di comunicazione sincrona tra due processi permettendo anche la

condivisione di oggetti. In ambito di sicurezza, grazie al Binder IPC Driver, si permette ai processi, che sono isolati all'interno della propria sandbox, di poter comunque comunicare in maniera sincrona

6. Elencare e descrivere i principali metodi callback previsti dall'interfaccia Service Android.

I metodi di callback previsti dall'interfaccia dei Service Android sono:

onCreate() crea un nuovo service

onDestroy() termina e dealloca un service

onStartCommand permette di richiamare una operazione da eseguire in background

onBind() mette in comunicazione il service e l'app che richiede quel service

7. Descrivere il funzionamento del Content Provider in Android. Indicare inoltre la struttura del relativo URI pubblico.

Il content provider permette alle app di accedere ai dati, i dati vengono restituiti come tabelle di un database relazionale, l'accesso alle singole istanze di queste tabelle viene fatto mediante un cursore.

Un content provider è identificato dal suo URI strutturato in 4 parti (A://B/C/D)

A. Protocollo (content)

B. Package del content provider

C. Tabella

D. Istanza

Esempio: content://contacts/people/2

8. Descrivere l'utilità dei tag <permission> e <uses-permission> nella configurazione di un app Android.

Tag XML presenti nel file AndroidManifest.xml

<permission> definisce un permesso custom che un'applicazione esterna deve richiedere

<uses-permission> richiede l'utilizzo di un permesso esterno all'app

9. Spiegare sinteticamente il meccanismo di comunicazione mediante Binder su Android.

Il Binder IPC di Android è un metodo di comunicazione tra processi sincrono in cui viene stabilito un canale di comunicazione all'interno del quale possono essere condivisi degli oggetti. Ad ogni app viene messo a disposizione un pool di threads per la gestione di questo meccanismo di comunicazione.

10. Indicare le differenze fra una Activity e un Service nel SO Android.

Un'activity è una schermata di un app, e interagisce direttamente con l'utente in quanto è attiva quando è in

foreground

Un service è un oggetto di un'app che viene eseguito in background senza interazione con l'utente

11. Descrivere sinteticamente le due principali tipologie di intent in Android.

Un intent è un'azione che coinvolge più componenti di un app, può essere:

- Esplicito: azione che individua univocamente un destinatario (es. aprire il menu delle impostazioni della propria app)
- Implicito: azione generica che non individua un destinatario (es. aprire una pagina web senza specificare con quale browser)

12. Descrivere sinteticamente il funzionamento del componente Broadcast Receiver in Android.

Un broadcast receiver è un'entità passiva di un app, è in ascolto del verificarsi di eventi, l'unico metodo di callback che possiede è `onReceive()` e per la sua esecuzione non richiede necessariamente che la relativa app sia in esecuzione.

13. Cosa si intende per Visible Lifetime e Foreground Lifetime in Android?

- Visible lifetime: da `onStart()` a `onStop()` rappresenta il tempo in cui un'activity è visibile all'utente
- Foreground lifetime: da `onResume()` a `onPause` rappresenta il tempo in cui l'activity è visibile e l'utente ci può interagire.

14. Descrivere le caratteristiche di un Service Android.

Un service Android è uno delle componenti di un'app e come tale costituisce un entry point dell'app. Rappresenta delle operazioni senza interfaccia grafica che vengono eseguite in background. I metodi per la gestione del ciclo di vita di un service sono:

- `onCreate()` inizializza un service
- `OnDestroy()` termina e dealloca un service
- `OnStartCommand()` richiama un'operazione da eseguire in background
- `OnBind()` mette in comunicazione il service con il processo che lo richiede

15. Si spieghi sinteticamente il funzionamento del Power Manager di Android.

Il Power Manager di Android applica, oltre ai meccanismi di risparmio energetico ereditati da Linux, un meccanismo di gestione dell'energia del dispositivo basata su wakelocks. Un app, per mantenere attiva una risorsa deve fare richiesta esplicita di un opportuno wakelock, in assenza del quale android spegne la risorsa per risparmiare energia.

16. Si illustri il ciclo di vita di una Activity in Android.

Il ciclo di vita di un'Activity ha tre stati:

- Attiva, ossia completamente visibile all'utente che può interagire con essa
- In pausa, ossia parzialmente oscurata (es. notifica push) l'utente non può interagire
- Terminata, chiusa e non più visibile all'utente

17. Spiegare ruolo e funzione dello strato JNI nello stack Android.

Il Java Native Interface fornisce un'interfaccia che permette alle funzionalità di alto livello di comunicare e richiamare librerie e funzionalità native.

18. Indicare e spiegare i meccanismi previsti dal sistema operativo Android per migliorare la durata della batteria di uno smartphone.

Android, oltre ai meccanismi di risparmio energetico ereditati da Linux, aggiunge un meccanismo di risparmio energetico basato sui wakelocks. Un'applicazione, per poter mantenere attiva una risorsa deve fare richiesta esplicita di un opportuno wakelock in assenza del quale Android disattiva la risorsa spegnendola per risparmiare energia.

19. Spiegare sinteticamente i meccanismi di block erasing e memory wearing all'interno del SO Android.

Il block erasing è una caratteristica delle flash memory, consiste nel fatto che bisogna cancellare una cella di memoria prima che vi sia consentita una scrittura, questo richiede molto tempo quindi viene fatto in tempi di idle. Il memory wearing è un'altra caratteristica delle flash memory e consiste nel distribuire su più celle di memoria le operazioni di lettura e scrittura in quanto se fatte sempre sulle stesse celle di memoria ne si provoca il deterioramento. Ogni cella di memoria ha un numero limitato di scritture che si possono fare.

20. Descrivere il meccanismo adottato dal S.O Android per gestire le situazioni di carico elevato della memoria RAM del dispositivo.

Nel momento in cui la RAM risulta essere sovraccaricata, il demone Lmkd (Low Memory Killer Daemon) controlla quale processo sta esercitando più "pressione" sulla RAM (di fatto rallentando il sistema). A quel punto il segnale vmpressure termina il processo in questione.

21. Descrivere sinteticamente le peculiarità di Dalvik VM. Indicare inoltre le differenze con ART.

La Dalvik VM è caratterizzata da tre importanti obiettivi progettuali:

- deve girare su dispositivi con poca memoria RAM e processori meno performanti;
- non utilizza spazio di swap a causa della ridotta memoria disponibile;
- deve girare su dispositivi dotati di scarsa capacità energetica.

Il codice in formato .dex viene runnato in due modi diversi: la parte di codice più utilizzata (Hot Code) viene compilato Just-In-Time; il resto (Cold Code) viene interpretato. ART (Android RunTime), la nuova VM che

sostituisce totalmente Dalvik a partire da Android 5.0, si basa sulla tecnologia AOT (Ahead-Of-Time), attraverso cui l'app viene interamente compilata in codice macchina nativo durante l'installazione sul device, e non durante l'esecuzione. Così facendo il codice compilato dell'applicazione è direttamente eseguibile, riducendo notevolmente i tempi di esecuzione rispetto al compilatore JIT usato in Dalvik, generando anche un certo risparmio energetico. Di contro aumenta il consumo di spazio sulla memoria sul dispositivo e il tempo di installazione dell'app.

22. Descrivere ruolo e funzionamento del processo ZYGOTE di Android

All'avvio del sistema init avvia Zygote, il cui scopo è quello di avviare la prima istanza di ART e crearne una nuova ogni qualvolta viene avviato un nuovo processo, eseguendo una fork() sulla prima istanza; oltre a questo precarica le java lib condivise ed avvia il system server.

23. Descrivere in dettaglio le differenze fra i due formati .dex e .oat nelle applicazioni Android

Gli eseguibili in formato .dex sono caratterizzati dal codice diviso in Hot Code (codice utilizzato più frequentemente, compilato Just In Time) e Cold Code (interpretato). Gli eseguibili in formato .oat invece prevedono che tutto il codice sia compilato Ahead Of Time (al momento dell'installazione).

24. Descrivere sinteticamente i componenti di architettura S.O Android. Specificando quali componenti sono stati modificati o sostituiti rispetto al Kernel Linux e spiegarne il motivo.

Android è un sistema open source per dispositivi mobili. Il suo stack software è costituito da un sistema operativo Linux-based. Le funzionalità aggiunte sono: -Power Management: fornisce un servizio di gestione energetica al di sopra di quello standard presente nei sistemi linux attraverso i wakelocks; - Binder (InterProcessCommunication) driver: permette la comunicazione tra processi (IPC) sincrona con un basso costo computazionale e quindi un basso consumo di batteria; - Low memory killer: Il demone Imkd misura i ritardi nell'esecuzione dei task dovuti alla mancanza di memoria e il segnale vmpressure, generato dal kernel, è utilizzato per scegliere quali task killare per alleviare la pressione. Android eredita da Linux il directory tree ma utilizza un file system caratterizzato da no seek time, wear leveling, block erasing.

iOS

1. Descrivere in modo schematico ma dettagliato le funzionalità gestite dal componente BSD del Kernel XNU di iOS.

BSD è il componente monolitico del kernel ibrido XNU di iOS, le funzionalità offerte da BSD sono:

- Protezione della memoria mediante mappatura e protezione delle pagine.
- Modello di sicurezza di UNIX con permessi, utenti, gruppi, ecc...
- Modello dei processi basato su processID, utenti proprietari, gruppi proprietari, segnali
- API POSIX e syscall
- Networking con protocolli TCP/IP, socket, firewall, ecc...
- Thread lato utente (pthread) basati sui Mach thread

2. Descrivere il modello MVC adottato da iOS per il design delle applicazioni. Chiarire in cosa esso si differenzia rispetto al modello MVC tradizionale.

Il modello MVC adottato da iOS è il Cocoa MVC, esso è composto da tre componenti:

- Model: contiene i dati dell'app e la logica con cui vengono manipolati questi dati
- View: rappresenta un rettangolo sullo schermo del dispositivo renderizzato ed animato da CoreAnimation
- Controller: funge da intermediario tra Model e View, modifica le gerarchie di view a seguito di modifiche sui dati del model, inoltre gestisce le interazioni dell'utente con le view coordinandosi anche con altri controller per portare a termine tasks complessi.

La differenza sostanziale con il modello MVC tradizionale è che nel Cocoa MVC adottato da iOS Model e View non sono strettamente accoppiati ma comunicano tra loro mediante il controller che fa da intermediario, questo favorisce il riuso delle view per le app, cosa che non è possibile nel modello MVC tradizionale in cui view e model sono strettamente accoppiati

3. Descrivere il funzionamento del Secure Boot Chain di iOS.

L'avvio di iOS avviene per diversi stadi, ognuno dei quali è crittograficamente verificato dallo stadio precedente, questo funzionamento è chiamato Boot Chain of trust. I diversi stadi sono dislocati in memorie differenti, queste memorie sono:

- SecureROM: memoria read-only inaccessibile dal kernel
- NAND Flash: memoria r/w divisa in due partizioni:
 - o BOOT: memoria inaccessibile ai processi e al kernel e sovrascrivibile solamente mediante aggiornamenti software
 - o FSYS: contiene il file system, visibile al kernel e ai processi utente

I vari stadi del Secure Boot Chain sono:

- BootROM: residente nel SecureROOM, possiede la fiducia intrinseca del sistema, viene scritto in fase di produzione e contiene le primissime istruzioni svolte per l'avvio del sistema, contiene la root CA key di Apple con cui vengono verificati crittograficamente gli stadi successivi.

- LLB (Low Level Bootloader): componente opzionale in quanto non più presente nelle versioni successive all'iPhone 6s, residente nella partizione BOOT della memoria NAND Flash, contiene delle routine di avvio del sistema e verifica crittograficamente lo stadio successivo.

- iBoot: residente anch'esso nella partizione BOOT della NAND Flash, contiene delle istruzioni di avvio di alto livello, localizza il kernel, ne verifica l'integrità, lo carica in memoria e ne lancia l'esecuzione.

Se si verifica un fallimento a livello di BootROM→LLB→iBoot il sistema entra in modalità di Device Firmware Update

Se si verifica un fallimento a livello di iBoot→kernel il sistema entra in modalità di recovery

4. Indicare le funzionalità gestite da ciascuna componente del kernel ibrido di iOS

iOS ha il kernel ibrido XNU, questo kernel è composto da Match (microkernel) e BSD (monolitico)

Le funzionalità offerte da queste due componenti sono:

Match:

- CPU scheduling basata su preemption e code a priorità
- IPC basata su message passing
- RPC basate su Match IPC
- Thread lato kernel
- Memoria virtuale paginata
- Supporto soft real time

BSD:

- Protezione della memoria mediante mapping e protezione delle pagine
- Modello di sicurezza di UNIX con proprietari, gruppi, permessi, policy, ecc...
- Modello dei processi con PID, utenti proprietari, gruppi proprietari, segnali, ecc..
- Supporto al networking con protocolli TCP/IP, socket, firewall, ecc..
- API POSIX e syscall
- Thread lato utente (pthread) basati sui Match thread
- Virtual file system BSD con filesystem fisici come FAT32

5. Si descrivano le policy di sicurezza per la protezione del kernel iOS.

Per proteggere il kernel iOS adotta le seguenti policy di sicurezza:

- Write XOR Execute: ossia imposta sulle pagine di memoria o il permesso di scrittura oppure quello di esecuzione, mai entrambi insieme.
- KASLR (Kernel Address Space Layer Randomization), consiste nel caricare il Kernel in posizioni di memoria differente ad ogni avvio del sistema.
- KPP (Kernel Patch Protection), consiste in un check sia hardware che software del kernel, in caso di anomalie il sistema entra in kernel panic
- Pointer Authentication, i puntatori critici alla memoria vengono protetti tramite autenticazione crittografica via hardware

6. Descrivere le caratteristiche principali del pattern MVC in iOS.

Il Pattern MVC in iOS è scomposto da tre componenti:

- Model: contiene i dati dell'app e la logica con cui vengono modellati
- View: rappresenta un rettangolo sullo schermo del device renderizzato ed animato da CoreAnimation
- Controller: intermediario tra model e view, modifica le gerarchie di view a seguito di modifiche ai dati del model, gestisce le interazioni dell'utente con le view coordinandosi con altri controller per portare a termine tasks complessi.

Grazie al controller model e view non sono strettamente collegati e questo favorisce il riutilizzo delle view

7. Descrivere le politiche di sicurezza adottate dal kernel iOS

Le politiche di sicurezza adottate da iOS per la protezione del kernel sono:

- Non concedere mai insieme i permessi di scrittura ed esecuzione delle pagine di memoria
- KASLR (Kernel Address Space Layout Randomization) caricare il kernel in posizioni diverse di memoria ad ogni avvio
- KPP (kernel Patch Protection) check sia hardware che software dekk'integrità del kernel, in caso di anomalie si va in kernel panic.
- Pointer authentication: Puntatori critici alla memoria sono proteti mediante autenticazione crittografica via hardware.

8. Si descrivano le peculiarità generali del sistema operativo iOS.

IOS è un SO proprietario di Apple progettato per girare su dispositivi mobile, ossia iPone e nelle varianti WatchOS e TVOS gira rispettivamente su AppleWatch e AppleTV, è closed source ma con diverse componenti open source. Come Mac OS è basato su Darwin che è un SO completo open Source con kernel XNU che è un kernel ibrido con parte monolitica BSD e parte a micro kernel Match. Con SWIFT è possibile sviluppare app native per iOS.

È un SO layerizzato:

- CocoaTouch

- CoreMedia
- CoreService
- CoreOS
- Kernel XNU e driver

Sicurezza di avvio con secure boot chain a stadi.

Meccanismi di sicurezza per la protezione del kernel, i dati, le app, ecc...

Dati biometrici salvati nel SEP.

Tutto è un app anche l'interfaccia grafica springboard.

La distribuzione dei software è centralizzata (appstore)

9. Apple File System, caratteristiche principali, prestazioni e sicurezza anche in rapporto con altri file system noti

Il FS proprietario di Apple supporta inode numbers a 64 bit; utilizza il meccanismo del Copy On Write (i file condivisi vengono ripetuti solo se modificati), e grazie a questo meccanismo crea periodicamente una snapshot del FS per ripristinarlo in caso di crash; supporta partizioni a dimensione dinamica e protegge crittograficamente i file con una o più chiavi.

10. Descrivere come iOS attua politiche di sicurezza per i dati e applicazioni

Per quanto riguarda la sicurezza dei dati, iOS sfrutta un co-processore per la crittografia, cifrando usando una o più chiavi uniche ogni file; inoltre i dati biometrici sono salvati in un sottosistema hardware (SEP, Secure Enclaver Processor) inaccessibile a qualunque processo, kernel incluso. Mentre per quanto riguarda la sicurezza delle applicazioni, in iOS è presente il code signing ovvero ogni app dev'essere firmata crittograficamente da Apple per poter essere lanciate; oltre al sandboxing utilizzato per evitare che ogni app non acceda a file di altre app o a feature alle quali non ha autorizzazione

11. Descrivere brevemente le caratteristiche del filesystem adottato da iOS.

Il filesystem adottato da Apple è APFS (Apple File System) ed offre:

- Inode numbers a 64 bit
- CoW (Copy On Write): quando un file viene copiato, la sua memoria non viene duplicata.
- Snapshots: APFS può fare istantanee dei suoi contenuti, in modo da poter ripristinare file o interi volumi ad uno stato noto.

Grazie alla CoW, gli snapshot non utilizzano spazio aggiuntivo al momento della loro creazione.

- Supporto a partizioni di dimensione dinamica tramite condivisione di «container» di dimensione fissa.
- Crittografia: cifratura a chiave singola univoca per file, o a chiave multipla per cifrare diverse parti dei file con una chiave diversa

NuttX/PX4

1. Descrivere i principali algoritmi di scheduling della CPU adottati dai SO real time. Specificare quali fra essi è impiegato nel SO NuttX. Chiarire in dettaglio le motivazioni della scelta.

I principali algoritmi di CPU scheduling per sistemi real-time sono:

- Round Robin: basato su preemption, viene concesso al processo in esecuzione un quanto di tempo di CPU (time slice) allo scadere del quale viene sottratta la CPU al processo in esecuzione e viene schedulato un nuovo processo.
- Schedulazione a priorità: processi con priorità più alta vengono schedulati prima di processi con priorità più bassa, solitamente i processi con la priorità più alta sono quelli real-time, questo algoritmo può essere combinato con algoritmi basati su preemption.
- Rate monotonic scheduling: priorità statica per ogni processo calcolata in base alla frequenza di utilizzo della CPU; alta frequenza di utilizzo → alta priorità; si può applicare la preemption NuttX. Schedula i suoi task con l'algoritmo di scheduling rate monotonic scheduling, questo perché è l'unico algoritmo in grado di rispettare le deadline imposte da un sistema real time come NuttX, cosa che infatti non è garantita nel Round Robin e nella schedulazione a priorità.

2. Descrivere brevemente l'architettura del flight stack PX4.

L'architettura di PX4 si divide in Flight stack e Middleware.

L'architettura del Flight stack a sua volta si divide in:

- Estimator: riceve in input i dati dai sensori di bordo del device, li combina e li elabora definendo lo stato attuale del device e fissa un setpoint che invia al controller.
- Controller: riceve in input una stima dello stato del device e il setpoint dall'estimator, in base a questi invia dati e comandi al device per effettuare delle correzioni cambiandone l'assetto e muovendolo verso la posizione desiderata.
- Mixer: riceve in input i dati e i comandi dal controller e li traduce in comandi low-level da inviare ai singoli motori del device assicurandosi che questi ultimi nel rispondere ai comandi rispettino dei limiti strutturali prestabiliti.

3. Descrivere gli stati previsti dallo scheduling dei task su NuttX. Durante lo scheduling dei task, i task, in base al loro stato si muovono su diverse liste durante il loro ciclo di vita:

- Inactive_tasks: lista di task non ancora avviati o terminati.
- Ready-to-run: lista dei task inizializzati e pronti ad essere eseguiti, in testa alla lista c'è il task attualmente in esecuzione e in coda ci sono i tasks in idle.
- Pending_tasks: lista dei tasks pronti a partire ma bloccati perché sul task attualmente in esecuzione non è applicabile la preemption, oppure perché i task in coda alla lista ready-to-run hanno priorità maggiore.
- Blocked_tasks: in questo stato ci sono due liste di tasks: waitingforsemaphore e waitingforsignal

4. Descrivere il concetto di task in NuttX, chiarendo le eventuali analogie e differenze con processi e thread di un sistema operativo general purpose.

Un task in NuttX è una sequenza di istruzioni dotata di un opportuno stack. Ogni task è dotato di un taskID e un Task Control Block che contiene i metadati e le informazioni circa il task, in analogia ai processi di un SO general purpose che posseggono un PID e un PCB con le stesse funzioni. Un task in NuttX, così come un processo in un SO general purpose, può istanziare uno o più thread. La gerarchia dei task in NuttX è di tipo parent/child in cui un child task eredita dal parent task i file stream stdin e stdout; il parent task attende che il child task termini per essere rimosso. Questo è abbastanza analogo alla gerarchia padre/figlio dei processi general purpose, solo che in aggiunta al padre che attende la terminazione del figlio per essere rimosso, nei processi general purpose c'è la possibilità che un processo padre entri in concorrenza con i processi figli per l'acquisizione delle varie risorse tra cui la CPU, inoltre un processo figlio può non condividere nessuna risorsa con il processo padre, può condividere un sottoinsieme di risorse del padre, oppure può essere un esatto duplicato del padre condividendo tutte le risorse. Altra differenza tra i task in NuttX e i processi nei SO general purpose è il tipo di scheduling, infatti nei SO general purpose ci sono tanti algoritmi di scheduling possibili tra cui FCFS, SJF, RR tutti combinabili con la preemption o no, in NuttX, per le necessità di un SO real time, si opta per il rate monotonic scheduling che assegna una priorità statica ad ogni processo in base alla frequenza di utilizzo della CPU (alta frequenza → alta priorità) abbinabile con la preemption, consente di rispettare le deadline di un SO real time come NuttX

5. Descrivere i principali meccanismi di IPC di NuttX.

I principali meccanismi di IPC in NuttX sono:

- Signals: ogni task, incluso l'interrupt handler può inviare dei segnali ad altri task, il task che riceve il segnale viene attivato e deve effettuare una routine di gestione del segnale ricevuto.
- POSIX Message queue: ogni thread è dotato di una coda il cui main è in attesa di messaggi l'interrupt handler invia un messaggio alla coda di un thread quando si verifica un evento, questo attiva il task ricevente che esegue l'elaborazione specifica

6. Descrivere come avviene il caricamento del firmware di PX4 su una piattaforma.

Il firmware di PX4 viene caricato mediante cross-compilazione, ossia il codice del firmware viene compilato a partire da file sorgenti su una piattaforma con architettura differente rispetto alla piattaforma su cui il codice compilato verrà eseguito. Questo viene fatto con il tool make con i makefile che definiscono le regole per il caricamento dei moduli compilati.

Esempio:

make px4_fmu-v2_default indica che sto compilando il target px4_fmu-v2 che rappresenta la versione del firmware scelto in base all'hardware su cui dovrà girare e default indica la configurazione scelta. Per caricare il codice compilato sulla piattaforma su cui desideriamo runnarlo (es. drone) si utilizza il comando make px4_fmu-v2_default upload. Il firmware di px4 è estendibile con dei moduli e programmi esterni, questi vengono inclusi nella directory src/examples insieme al file CmakeList.txt che contiene le indicazioni per la compilazione dei codici aggiunti. Successivamente occorre anche indicare all'interno del file default.cmake presente nella directory /boards/px4/fmu-v2 le applicazioni che desidero includere. Grazie a questa possibilità posso rendere il software PX4 intelligente con degli algoritmi di intelligenza artificiale rendendo l'autopilota ancora più indipendente da interventi umani.

7. Spiegare le principali differenze tra un sistema operativo hard e soft real-time. Indicare a quale delle due tipologie appartiene il S.O. NuttX e perchè.

I sistemi soft real time si differenziano dai sistemi hard-real time in quanto possono anche non rispettare la deadline, tuttavia con il passare del tempo il risultato diventa sempre meno accettabile. Il sistema Operativo NuttX è hard real time in quanto deve rispondere dinamicamente ed in tempi brevi agli eventi. Spiegare la struttura generale dei driver del S.O. NuttX

8. Spiegare la struttura generale dei driver del S.O. NuttX

I driver in Nuttx si dividono in 2 livelli:

-Upper half driver: sono indipendenti dall'architettura HW, implementa la logica di gestione del dispositivo, è implementato a livello utente;

-Lower half driver: sono collegati agli upper half con interfacce comuni ma allo stesso tempo con l'HW utilizzando delle interfacce specifiche, implementate nel kernel;

9. Descrivere i principali meccanismi di IPC di NuttX

I tre principali meccanismi di IPC in nuttx sono:

- signals: un task invia un segnale ad un altro task
- posix message queue: i task comunicano tra loro tramite code, con tipologie uniche di messaggio
- semafori: in cui, con la mutua esclusione, il semaforo è inizializzato ad 1 e ciò implica che la risorsa è impegnata. Mentre con signaling semaphores, il semaforo è inizializzato a 0 e un altro task lo inizializza a 1 indicando che la risorsa è disponibile.

ROS

1. Descrivere il meccanismo di comunicazione adottato da ROS per l'interazione asincrona fra diversi nodi. Spiegare i vantaggi di tale meccanismo rispetto alla comunicazione sincrona client-server

I nodi in ROS comunicano tra loro mediante il paradigma publish/subscribe asincrono, il funzionamento di questo paradigma consiste in uno nodo publisher che pubblica il messaggio che desidera inviare all'interno di un topic e un nodo subscriber si sottoscrive a quel topic per poter leggere le informazioni di quel messaggio. Un topic è un flusso di dati molti a molti, fortemente tipizzato dal tipo di messaggi pubblicati al suo interno. Più nodi publishers pubblicano e più nodi subscribers si sottoscrivono ad uno o più topics. Un topic quindi permette l'interazione asincrona tra due nodi senza che questi interagiscano in maniera diretta tra loro. Il vantaggio è un isolamento maggiore dei nodi, i quali non devono attendere risposte da altri nodi e quindi continuano la loro esecuzione, mentre nel modello client/server sincrono il client che richiede un servizio deve attendere, sospendendo la sua esecuzione, che il server glielo fornisca.

2. Spiegare il ruolo dei messaggi nella comunicazione publish/subscribe in ROS e indicare il numero massimo di tipi di messaggio che possono essere associati ad un singolo topic. Scrivere il comando da terminale per mostrare il tipo di messaggio di uno specifico topic.

I messaggi all'interno della comunicazione publish/subscribe in ROS, sono una struttura dati con la funzione di contenere le informazioni che i nodi publisher desiderano pubblicare in un topic, essi sono caratterizzati da avere un tipo. Un topic è fortemente tipizzato dal tipo di messaggi che contiene, quindi all'interno di un singolo topic possono essere pubblicati solamente un tipo di messaggi.

Per mostrare il tipo di messaggio di uno specifico topic basta quindi mostrare il tipo di topic con il comando `$rostopic type [nome_topic]`.

Mentre per mostrare la struttura di un determinato messaggio si utilizza il comando `$rosmmsg show [msg]` che appunto mostra la struttura con cui è composto il messaggio passato come parametro.

3. Spiegare il funzionamento dei comandi `roscore` e `roslaunch` su ROS.

Il comando `$roslaunch` avvia il processo core di ROS il quale avvia diversi nodi e processi essenziali tra cui:

- ROS Master
- ROS Parameter Server
- Rosout Logging Node

Il comando `$roslaunch` con la seguente sintassi: `$roslaunch PACKAGE EXECUTABLE` localizza e avvia il file eseguibile di nome EXECUTABLE all'interno del package di nome PACKAGE.

4. Spiegare il funzionamento del nodo master in ROS. Indicare il comando per avviarlo da terminale.

Il nodo master in ROS è uno dei nodi presenti all'interno del nodo core di ROS e offre un meccanismo di lookup tra i nodi permettendogli quindi di comunicare tra loro dandogli la possibilità di trovarsi all'interno dell'architettura distribuita peer-to-peer con cui sono organizzati i nodi in ROS. Il nodo master, insieme al ROS Parameter Server e al rosout logging node, viene lanciato con il comando da terminale `$roslaunch`

5. Spiegare il funzionamento delle callback in ROS. Ci si aiuti con degli esempi.

Il modo con cui ROS risolve l'avvenire di eventi asincroni che danno fastidio all'elaborazione sequenziale dei software è mediante le callback, esse sono delle funzioni che gestiscono appunto questi eventi asincroni e vengono richiamate ogni volta che sono disponibili i dati per l'elaborazione. Un esempio può essere la gestione dell'object-avoidance. I dati restituiti dai sensori presenti nell'hardware del robot in questione rilevano un ostacolo, mediante il richiamo di una opportuna callback passandogli in input i dati rilevati dai sensori del robot, questa riesce a gestire questo evento e a portare il robot ad evitare l'ostacolo. (????)

6. Descrivere sinteticamente il funzionamento del nodo master in ROS. Indicare inoltre i comandi per avviare il nodo master ed elencare i topic publisher/subscriber.

Il nodo master in ROS si occupa di stabilire una connessione per la comunicazione tra i nodi, inoltre offre un meccanismo di lookup che permette ai nodi di trovarsi vicendevolmente, senza di esso i nodi non potrebbero comunicare.

\$roscore avvia il nodo Master oltre che il ROS Parameter Server e il rosout logging node.

\$rostopic list mostra tutti i topic attivi nel sistema.

\$rosgrep info nome_nodo mostra informazioni sul nodo passato come parametro tra cui i topic su cui pubblica (publisher) e i topic a cui è sottoscritto (subscriber)

7. Si descrivano sinteticamente i due principali meccanismi di comunicazione IPC tra i nodi in ROS.

I due principali meccanismi di IPC tra i nodi in ROS sono:

- Modello publish/subscribe asincrono: nodi publisher pubblicano messaggi su topic e nodi subscribe che sono sottoscritti al topic leggono i messaggi
- Modello sincrono mediante i ROS Service: nodo richiede un servizio ad un altro nodo ed attende che quest'ultimo glielo fornisca, comunicazione mediante una coppia di messaggi strutturati, uno di richiesta di servizio, l'altro di risposta.

8. Si illustrino le funzionalità del nodo ROS master. Specificare il comando per avviarne l'esecuzione.

Il nodo master fornisce una connessione per la comunicazione tra i ROS node, inoltre fornisce un meccanismo di lookup che permette ai nodi di localizzarsi a vicenda per poter comunicare. Il comando che lancia il ROS master insieme ad altri nodi e programmi è: \$roscore

9. Spiegare cosa si intende per tipizzazione dei topic in ROS.

Un topic è fortemente tipizzato dal tipo di messaggi pubblicati al suo interno, questo implica che, definito il tipo di messaggi che si intende pubblicare su un topic, questo non può contenere tipi di messaggi diversi dal tipo di messaggio su cui è definito.

10. Rappresentare schematicamente la comunicazione tra un nodo publisher e un nodo subscriber in un sistema ROS-based. Specificare il ruolo del nodo Master in tale comunicazione.

Nodo publisher e nodo subscriber comunicano attraverso un topic:

- Publisher notifica al ROS Manager che vuole pubblicare un messaggio su un determinato topic
- ROS Master controlla che il topic abbia almeno un subscriber, se non ne ha, nessun messaggio viene pubblicato
- Subscriber notifica al ROS Master che vuole sottoscrivere ad un determinato topic
- Se un topic ha sia un publisher che un subscriber ROS Master permette al nodo publisher di pubblicare il messaggio e quindi al nodo subscriber di leggerlo.

11. Si descrivano le peculiarità principali del sistema ROS.

ROS è un framework per lo sviluppo di software per sistemi robotici. È chiamato meta-sistema operativo in quanto offre delle funzioni comuni a tutti i sistemi operativi tra cui:

- Astrazione dell'hardware
- Meccanismi di IPC(message passing)
- Sviluppo di funzionalità commonly-used
- Controllo low-level dei device
- Gestione dei pacchetti

ROS è indipendente dai linguaggi di programmazione ed è scalabile su più piattaforme. Ogni funzionalità offerta da ROS è gestita da un singolo processo, i processi sono detti Nodi e comunicano tra loro mediante un paradigma di comunicazione asincrono detto publish/subscribe, ossia nodi publisher pubblicano messaggi su topics i cui nodi sottoscritti detti appunto nodi subscribers leggono i messaggi. ROS è formato dal ROS core che contiene un insieme di nodi e programmi essenziali tra cui:

- ROS master
- ROS Parameter Server
- Rosout logging node

Permette di comandare da riga di comando o mediante joystick dei robot reali oppure, mediante dei tool simulativi come turtlesim, dei robot virtuali.

12. Si illustrino le funzionalità del nodo ROS Master.

Il nodo ROS Master si occupa di stabilire la connessione per la comunicazione tra i ROS node fornendo un meccanismo di lookup che permette ai nodi di trovarsi a vicenda.

13. Si descrivano le caratteristiche generali di ROS chiarendone peculiarità e potenzialità.

ROS è un framework per lo sviluppo di software per sistemi robotici, fornendo API e funzionalità comuni ai SO, per questo è chiamato meta sistema operativo, tra le funzionalità ci sono:

- Astrazione dell'hardware

- Meccanismi di IPC
- Callback
- Controllo low-level di device

I processi in ROS sono visti come nodi di un'architettura distribuita, permettendo un isolamento tra essi facendo in modo da assegnare un processo per ogni funzionalità risolvendo la complessità dell'implementazione di software per sistemi robotici.

Il ROS core contiene un insieme di nodi e programmi essenziali tra cui:

- ROS Master
- ROS Parameter Server
- Rosout logging node

14. Si spieghi il meccanismo di comunicazione interprocesso adoperato in ROS chiarendo brevemente a quale categoria di IPC appartiene.

Il meccanismo di IPC in nuttx è il modello publish/subscriber in cui nodi publisher pubblicano messaggi su dei topic e i nodi subscriber sottoscritti a quel topic possono leggere i messaggi al suo interno. Questo meccanismo di IPC appartiene alla categoria di IPC basati su message passing.

15. Si spieghi il meccanismo di storage e retrieval dei parametri a runtime previsto da ROS. Aiutarsi con uno schema.

I parametri in ROS vengono gestiti dal ROS Parameter Server. Il ROS Parameter Server offre un dizionario condiviso di parametri, immagazzinati con delle chiavi in una locazione centrale. I parametri vengono recuperati a runtime usando delle chiavi.

16. Si spieghi il meccanismo di comunicazione publish/subscribe utilizzato in ROS.

Il meccanismo di comunicazione publish/subscribe è un meccanismo di IPC asincrono basato su message passing, esso prevede:

- Nodi publisher: pubblicano messaggi su dei topic
- Nodi subscriber: effettuano una sottoscrizione ai topic che contengono i messaggi di loro interesse
- Topic: flusso di comunicazione molto a molti che separa le sorgenti di informazioni dai destinatari che le ricevono.

17. Quale funzione riveste il ROS Parameter Server? Si indichi almeno un comando per interagire con tale componente.

Il ROS Parameter Server offre un dizionario condiviso di parametri di configurazione e altri dati. I parametri sono nominati usando la normale convenzione di ROS. I parametri sono organizzati con una gerarchia per proteggere i nomi dei parametri da possibili collisioni, inoltre questo permettere di accedere ai parametri singolarmente o come struttura. Il ROS Parameter Server offre diversi tipi di dati per i parametri (es. int,

string, ecc...)

18. Spiegare il ruolo dei messaggi nella comunicazione publish/subscribe in ROS e indicare il numero trascritto di tipi di messaggi che possono essere associati ad un singolo topic. Scrivere: il comando da terminale per pubblicare messaggio “alarm” di tipo stringa su topic/events; il comando da terminale per mostrare il tipo di messaggio di uno specifico topic.

I messaggi all'interno della comunicazione publish/subscribe servono per tipizzare i topic, infatti ogni topic è fortemente tipizzato dai messaggi all'interno che sono di un unico tipo.

rostopic type mostra il tipo di messaggio di uno specifico topic; rostopic pub topic/events “alarm”.

19. Descrivere le caratteristiche dei service ROS, indicando il meccanismo IPC(Inter Process Communication) adottato. Scrivere inoltre il comando da terminale per elencare i service attivi e spiegarne la sintassi.

In ROS i nodi possono richiedere ad altri nodi un servizio secondo il paradigma request/reply (comunicazione sincrona). I service sono costituiti da una coppia di messaggi (uno per la request e uno per la reply). Il comando per visualizzare i service attivi è `$ rosservice list`.

20. Descrivere cos'è il Parameter Server di ROS, specificare i comandi per elencare tutti i parametri e per aggiornare il parametro background_color di valore 130.

Il Parameter Server di ROS fornisce un dizionario di parametri di configurazione condiviso e altri dati arbitrari da memorizzare e recuperare a runtime usando parametri chiave.

Il comando è: `rosparam set background_color 130`

Teoria

1. Con riferimento agli algoritmi di schedulazione della CPU, elencare quali sono i criteri generali di valutazione della bontà di uno schema e come sia possibile paragonare algoritmi diversi tra loro. (cap 6)

I criteri di valutazione e comparazione di algoritmi di scheduling diversi sono:

- Tempo di utilizzo della CPU
- Numero di processi completati nell'unità di tempo (throughput)
- Turnaround time : $t_{load} + t_{ready} + t_{CPU} + t_{I/O}$, gli algoritmi di scheduling agiscono su t_{ready}
- Tempo di attesa
- Tempo di risposta.

Un buon algoritmo di scheduling quindi massimizza l'utilizzo della CPU e il throughput mentre minimizza (di solito si minimizzano i valori medi) i tempi di attesa e risposta.

2. Descrivere il meccanismo dell'allocazione collegata (linked) di un file, evidenziando vantaggi e svantaggi rispetto al meccanismo di allocazione contigua. (cap 12)

L'allocazione collegata di un file consiste nel allocare ad un file dei blocchi di memoria non contigui ma collegati tra loro da puntatori come una lista concatenata, ogni blocco di memoria quindi contiene un puntatore al blocco di memoria successivo, la directory contiene un puntatore al blocco di memoria iniziale mentre il blocco di memoria finale ha un puntatore nullo in quanto non punta a nessun'altro blocco.

- Vantaggi: risolve i problemi incontrati nell'allocazione contigua, ossia il problema della frammentazione e la difficoltà di estendere la dimensione di un file
- Svantaggi: accessi diretti e casuali non più efficienti come nell'allocazione contigua in quanto richiede una scansione dei vari blocchi seguendo i puntatori, i puntatori occupano memoria aggiuntiva, una ricerca richiede diversi spostamenti della testina, in quanto i blocchi non sono contigui, rendendola onerosa.

3. Descrivere sinteticamente i tre modelli di programmazione multithread. Per ciascuno di essi chiarire pregi e difetti. Indicare anche quale dei modelli produce il maggiore impatto sulle prestazioni della macchina. (cap 5)

I tre modelli di programmazione multithread sono:

- Modello molti a uno: più user thread e un solo kernel thread, richiede l'uso di uno scheduler lato utente che schedula uno degli user thread da assegnare al kernel thread, il kernel non ha visione degli user thread.

Vantaggio: Più semplice da implementare

Svantaggi: no thread multipli in parallelo, chiamata bloccante da parte di un thread blocca l'intero processo

- Modello uno a uno: ogni user thread è associato ad uno ed uno solo kernel thread

Vantaggi: thread eseguibili in parallelo, chiamata bloccante di un thread non blocca l'intero processo.

Svantaggi: overhead della creazione dei kernel thread (viene posto un limite al multithreading).

- Modello molti a molti: n user thread e m kernel thread con $1 < m < n$.

Vantaggi: thread multipli in parallelo, non sovraccarica il sistema poiché crea un numero di kernel thread opportuno in base alle caratteristiche del sistema.

Svantaggio: Complessità dell'implementazione

Tra i tre modelli quello che ha un maggiore impatto sulle prestazioni della macchina è il modello multi a uno, in quanto una chiamata bloccante da parte di un thread blocca l'intero processo.

4. Si chiarisca il ruolo dello schedulatore a medio termine spiegandone inoltre le funzionalità. (cap 4)

Lo schedulatore a medio termine si occupa di diminuire temporaneamente il grado di multiprogrammazione del sistema andando a togliere momentaneamente uno o più processi dalla memoria centrale (swapping) alleggerendo il sistema.

5. Descrivere il problema della frammentazione (interna ed esterna) della memoria, con particolare riferimento allo schema di gestione che fa uso della segmentazione. Quali sono le differenze rispetto all'utilizzo della paginazione? (cap 9).

La frammentazione è un problema dell'allocazione dinamica della memoria ai processi, può essere:

- Frammentazione esterna: c'è abbastanza spazio di memoria per soddisfare la richiesta di un processo, ma questa non è contigua ma frammentata, quindi per soddisfare la richiesta del processo va effettuata una compattazione, cosa non banale se la rilocalizzazione degli indirizzi dei processi non è dinamica e fatta in tempo di esecuzione.
- Frammentazione interna: ad un processo viene allocata una porzione di memoria leggermente maggiore rispetto a quella di cui necessita, questo spazio di memoria in più risulterà inutilizzato e quindi sprecato.

La segmentazione è una tecnica di suddivisione logica dello spazio di indirizzamento di un processo in segmenti di codice di dimensioni diverse che hanno la stessa funzione (es. main, procedura, funzione, ecc...). Si allocano in memoria i singoli segmenti e non l'intero processo con gli algoritmi di allocazione dinamica (first fit, best fit, worst fit), poiché i segmenti non sono di dimensione fissa ma eterogenea si presenta ancora il problema della frammentazione esterna e questa è la differenza sostanziale con la paginazione. La paginazione, a differenza della segmentazione, consiste nel suddividere logicamente lo spazio di indirizzamento di un processo in pagine di dimensione fissa e suddividere l'intera memoria in frame della stessa dimensione delle pagine dei processi, l'allocazione riguarda quindi le singole pagine ed è un banale problema di assegnazione dei frame liberi della memoria alle pagine dei processi, in questo modo si elimina del tutto la frammentazione esterna, cosa che invece si presenta nella segmentazione, in quanto le pagine hanno tutte la stessa dimensione, tuttavia si presenta il problema della frammentazione interna quando l'intero spazio di indirizzamento del processo non è una potenza di 2, mediamente si ha per ogni processo mezza pagina inutilizzata per via della frammentazione interna.

6. Descrivere il meccanismo dell'allocazione collegata(linked) di un file, evidenziando vantaggi e svantaggi rispetto al meccanismo di allocazione contigua. (cap 12).

L'allocazione collegata dei blocchi di memoria ad un file consiste nel allocare i blocchi di memoria in maniera non contigua, ogni blocco contiene un puntatore al blocco successivo, i blocchi sono quindi sparsi

così come i puntatori. Vantaggio rispetto all'allocazione contigua è la possibilità di estendere facilmente la dimensione di un file nel tempo e l'eliminazione della frammentazione esterna

Svantaggio rispetto all'allocazione contigua è l'inefficienza nelle operazioni di ricerca e di accesso casuale, infatti per accedere ad un blocco bisogna seguire ordinatamente i vari puntatori con una scansione.

7. Si illustri il significato e le conseguenze della cosiddetta anomalia di Belady. Spiegare se ed eventualmente in che modo essa è superabile. (cap 10)

L'anomalia di Belady si presenta nell'algoritmo di sostituzione di pagine FIFO, esso avviene quando pur incrementando il numero di frame allocabili per processo, si verifica un inaspettato aumento delle page fault, questo fenomeno è completamente imprevedibile, per questo l'algoritmo FIFO non è adatto, l'unica soluzione è optare per un altro algoritmo di sostituzione di pagine come ad esempio LRU.

8. Si descriva brevemente il fenomeno di busy waiting nei semafori spiegandone l'origine e chiarendo in quale modo è possibile risolverlo. (cap 7)

Quando un thread T1 vuole entrare nella propria sezione critica ed effettua quindi un `acquire(S)`, ma un altro thread T2 è già all'interno della propria sezione critica quindi $S \leq 0$, allora T1 entra in un ciclo di attesa attivo fino a quando T2 non effettua un `release(S)`, ossia busy waiting, questo spreca cicli di CPU. Per risolverlo si introducono insieme ai metodi `acquire()` e `release()` i metodi `block()` e `wakeup()`:

- `Block()` richiamato dopo `acquire()` dal thread quando entra in un ciclo di attesa attivo, in modo da essere sospeso e rilasciare la CPU.
- `Wakeup()` richiamato da un altro thread quando lascia la sua sezione critica per svegliare il thread sospeso da `block()`.

9. Si descriva brevemente il fenomeno della starvation nell'ambito della schedulazione della coda di ready nel ciclo di esecuzione dei processi e si dica, motivando la risposta con quali approcci si può limitare un effetto del genere. (cap 6)

La starvation si verifica quando un processo attende indefinitamente l'allocazione della CPU, si verifica negli algoritmi basati su priorità sia con preemption che senza preemption, si verifica nel momento in cui un processo a bassa priorità attende indefinitamente l'allocazione della CPU poiché questa viene allocata sempre a processi con priorità più alta che continuano ad arrivare nella coda di ready. Una possibile soluzione è detta aging (invecchiamento), questa consiste nel far accrescere la priorità di un processo durante il suo tempo trascorso in attesa nella coda di ready, in questo modo si garantisce ad un processo che parte con una priorità bassa di acquisire la CPU dopo un periodo di tempo finito.

10. Spiegare sinteticamente il funzionamento della chiamata di sistema fork e indicare in quali casi può riscontrarsi la presenza di un processo zombie o un processo orfano. (cap. 4)

La chiamata di sistema `fork()` crea un processo figlio del processo padre che ha chiamato, il processo figlio inizialmente condividerà le strutture dati e lo spazio di indirizzamento con il padre.

Un processo zombie è un processo in fase di rimozione ma non del tutto rimosso, in cui non viene schedato ma è ancora presente nella tabella dei processi.

Un processo orfano è un processo figlio il cui processo padre è terminato, esso può essere adottato da init oppure a sua volta terminato.

11. Rappresentare schematicamente l'architettura di segmentazione con paginazione. Indicare se tale soluzione può generare frammentazione(interna/esterna) motivando la risposta. (cap 9)

La segmentazione paginata è una combinazione delle tecniche di segmentazione e paginazione. Consiste nel segmentare il processo in segmenti di codice logicamente affini, e nel suddividere lo spazio di indirizzamento del processo in due tabelle di segmenti:

- Tabella dei segmenti privati (local segment table) LST
- Tabella dei segmenti condivisi (global segment table) GST

Quindi l'indirizzo di un segmento è un indirizzo bidimensionale (selettore, displacement). Nel selettore sono contenute le informazioni circa il segmento tra cui :

- Id segmento nella Segment table
- 1 bit che indica in quale tabella si trova il segmento (LST o GST)
- 2 bit di protezione, indica quali permessi sono concessi per il segmento

Una volta acceduto all'elemento della specifica tabella dei segmenti grazie al selettore si ottengono i registri base e limite del segmento, sommando al registro base ottenuto il displacement dell'indirizzo di partenza si ottiene un indirizzo lineare ancora logico. Da qui parte la tecnica di paginazione, supponiamo la page table molto grande quindi paginazione a due livelli.

L'indirizzo logico ottenuto dalla risoluzione della segmentazione contiene: directory(equivalente a EPMT), pagina, displacement. Da qui, si risolve l'indirizzo logico in indirizzo fisico come in una paginazione a due livelli, ossia:

- Da directory risalgo all'elemento della tabella delle directory (equivalente a EPMT) da cui ricavo la page table.
- Dalla page table ricavata e da pagina dell'indirizzo logico ottengo l'elemento della page table della pagina specifica da cui ottengo il frame.
- Sommando il frame al displacement ottengo l'indirizzo fisico

Questo schema risolve la frammentazione esterna presente nello schema a segmentazione grazie alla paginazione che permette di avere pagine di dimensione fissa mappate in frame della stessa dimensione. La frammentazione interna è ancora non risolta come nello schema di paginazione, in quanto lo spazio di indirizzamento di un processo potrebbe non essere una potenza di 2.

12. Indicare le condizioni necessarie e sufficienti per identificare uno stato di deadlock. (cap 8)

Le condizioni necessarie e sufficienti per identificare uno stato di deadlock sono:

- Mutua esclusione: le risorse devono essere accessibili solo da un processo per volta.
- No preemption: non è possibile sottrarre ad un processo una risorsa che ha già acquisito.
- Hold and wait: un processo possiede delle risorse ed attende risorse in possesso da altri processi

- Circular wait: processo attende una risorsa in possesso di un altro processo che a sua volta attende una risorsa in possesso da un altro processo in maniera ciclica.

Queste 4 caratteristiche devono essere vere contemporaneamente affinché si possa verificare un deadlock.

13. Rappresentare schematicamente il funzionamento della schedulazione con coda multilivello retroazionata. Spiegare perché tale meccanismo previene la starvation dei processi. (cap 6)

La schedulazione con code multilivello retroazionate consiste nel dividere i processi in code multilivello, ciascuna coda ha un proprio algoritmo di scheduling, esiste anche uno scheduling delle code (code prioritarie, time slice), inoltre, a differenza della schedulazione con code multilivello non retroazionate, c'è la possibilità da parte dei processi di spostarsi tra le code, per questo retroazionate. Quest'ultima proprietà permette di risolvere il problema della starvation che si può presentare per la schedulazione con code multilivello, permettendo ad un processo di spostarsi in una coda che gode di priorità più alta o di un time slice più ampio.

14. Si descrivano brevemente le condizioni necessarie per il verificarsi di un deadlock. (cap 8)

Le condizioni necessarie affinché possa verificarsi il deadlock sono:

- Mutua esclusione: ogni risorsa è ad accesso esclusivo
- Possesso e attesa: un processo che possiede delle risorse ne attende delle altre che sono in possesso di altri processi
- No preemption: un processo rilascia le risorse solo di sua spontanea volontà, non li possono essere sottratte
- Attesa circolare: processo che attende una risorsa in possesso di un altro processo che a sua volta attende una risorsa in possesso di un altro processo e così via formando un ciclo di attesa

Queste 4 condizioni devono valere contemporaneamente.

15. Si indichi il ruolo svolto dal vettore delle interruzioni nel processo complessivo di gestione degli interrupt. (cap 2)

Il ruolo svolto dal vettore di interrupt è quello di contenere tutti gli indirizzi alle routine di gestione dei vari interrupt. Esso infatti è un vettore il cui indice rappresenta il codice di un determinato tipo di interrupt, il SO quando rileva un interrupt, attraverso il suo codice accede nel vettore di interrupt all'indirizzo di memoria che punta alle istruzioni della routine per la gestione del relativo interrupt.

16. Spiegare sinteticamente le problematiche associate al rilascio anticipato delle risorse nel caso di deadlock detection ad recovery. (cap 8)

Una volta rilevato un deadlock una possibile soluzione consiste nel rilascio anticipato delle risorse da parte di un processo, ossia effettuare una prelazione delle risorse, le problematiche relative a questa soluzione sono:

- Bisogna stabilire quali risorse e quali processi si devono sottoporre a prelazione considerando dei fattori di costo come numero di risorse occupate e tempo di elaborazione del processo.
- Bisogna effettuare un rollback del processo in modo da riportarlo ad uno stato sicuro
- Si può incontrare la starvation di un processo, in quanto può accadere che il rollback e il rilascio delle risorse vengano effettuate sempre allo stesso processo.

Una soluzione alla starvation consiste nel verificare che un processo possa subire la prelazione delle sue risorse un numero limitato di volte includendo il numero di rollback effettuati sul processo all'interno dei fattori di costo

17. Descrivere sinteticamente le tre principali tecniche di paginazione della memoria per architetture a 64 bit.(cap 9)

Le tre principali tecniche di paginazione della memoria per architetture a 64 bit sono:

- Tabella delle pagine con hashing. Consiste nell'utilizzo di una funzione di hashing che riceve in input il numero di pagina restituendo in output un indice i alla tabella delle pagine, questo indice i porta ad una lista concatenata in cui sono presenti tutte le pagine il cui numero ha portato ad un conflitto nella generazione dell'indice, scorrendo questa lista si trova l'elemento della pagina cercata che contiene il frame fisico, sommando al frame fisico il displacement dell'indirizzo logico iniziale si ottiene l'indirizzo fisico.
- Tabella delle pagine invertita. Ogni processo non ha più la propria tabella delle pagine ma c'è un'unica tabella delle pagine del sistema che memorizza tutte le pagine di tutti i processi. L'indice di questa tabella rappresenta il numero di frame fisico, per questo si dice invertita, mentre ogni elemento della tabella contiene il numero di pagina e il PID del processo a cui appartiene quella pagina.

La ricerca su questa tabella avviene per scorrimento in base al PID del processo e al numero di pagina, una volta trovato l'elemento cercato grazie all'indice di questo elemento si ottiene il frame fisico che sommato al displacement ci dà l'indirizzo fisico. Ricerca inefficiente quindi si opta per una mappatura dell'IPMT tramite hash table.

- Paginazione gerarchica, consiste nel paginare la PMT, che è troppo grande poiché contiene troppi elementi in un'architettura a 64 bit, creando una EPMT; poiché in un'architettura a 64 bit anche la EPMT è molto grande si può paginare anch'essa creando una paginazione a tre livelli. Un indirizzo è quindi formato da (p_3, p_2, p_1, d) in cui : 32 bit per p_3 , 10 bit per p_2 , 10 bit per p_1 e 12 bit per d con un totale di 64 bit di indirizzamento. Si nota che ancora la tabella delle pagine esterna ha troppi elementi (2^{32}) quindi si dovrebbe paginare ancora, ma nella risoluzione dell'indirizzo logico in fisico comporterebbe un accesso alla memoria per ogni livello di paginazione rendendo assai inefficiente l'accesso alla memoria fisica, per questo è meglio optare per le prime due tecniche.

18. Si descriva brevemente il fenomeno della starvation nell'ambito della schedulazione della coda di ready nel ciclo di esecuzione dei processi e si dica, motivando la risposta, con quali approcci si può limitare un effetto del genere. (cap 6)

La starvation di un o più processi avviene in ambito di scheduling su priorità e si ha quando processi di priorità bassa attendono indefinitamente l'allocazione della CPU perché un flusso continuo di processi a priorità più alta ne entrano continuamente in possesso. Per risolvere la starvation si può utilizzare un meccanismo di aging, consiste nel far acquistare priorità progressivamente al processo che attende in coda di ready assicurandosi in questo modo che dopo un tempo di attesa finito possa accedere alla CPU.

19. Si descriva brevemente il fenomeno di busy waiting nei semafori spiegando in qual modo è possibile risolverlo. (cap 7)

Quando un thread T1 vuole entrare nella propria sezione critica ed effettua quindi un acquire(S), ma un altro thread T2 è già all'interno della propria sezione critica quindi $S \leq 0$, allora T1 entra in un ciclo di attesa attivo fino a quando T2 non effettua un release(S), ossia busy waiting, questo spreca cicli di CPU.

Per risolverlo si introducono insieme ai metodi acquire() e release() i metodi block() e wakeup():

- Block() richiamato dopo acquire() dal thread quando entra in un ciclo di attesa attivo, in modo da essere sospeso e rilasciare la CPU.
- Wakeup() richiamato da un altro Thread quando lascia la sua sezione critica per svegliare il thread sospeso da block().

20. Si illustri l'implementazione dell'algoritmo di sostituzione delle pagine LRU mediante utilizzo dello stack (cap.10)

L'algoritmo di sostituzione delle pagine LRU può essere implementato mediante l'utilizzo dello stack. Le pagine all'interno dello stack sono messe in ordine di utilizzo, quella utilizzata più recentemente si trova in cima allo stack e via via scendendo nello stack troviamo quelle utilizzate meno recentemente. Quando viene fatta richiesta di una pagina si controlla se essa è presente all'interno dello stack, se è presente la pagina viene portata in cima allo stack in quanto è quella che è stata utilizzata più recentemente; se la pagina richiesta non è presente nello stack, allora si sceglie la pagina in fondo allo stack come vittima da sostituire e la pagina richiesta viene caricata in cima allo stack.

21. Si descriva brevemente il fenomeno della starvation nell'ambito della schedulazione della coda di ready nel ciclo di esecuzione dei processi e si dica -motivando la risposta- con quali approcci si può limitare un effetto di tal genere (cap 6)

22. Si descriva la serie esponenziale per la stima del next CPU burst in un algoritmo di scheduling SJF (cap. 6)

Per implementare l'algoritmo di scheduling SJF bisogna conoscere il next CPU burst, questo ovviamente non è possibile, per questo si effettua una stima mediante una media esponenziale con i seguenti parametri:

- alpha: coefficiente di peso, in base a questo coefficiente si decide se dare peso maggiore alla storia recente del processo o alla storia passata.
- τ_{n+1} è la stima del next CPU burst
- t_n è l'attuale CPU burst
- τ_0 è un parametro costante

la media esponenziale è la seguente:

$$\tau_{n+1} = \alpha * t_n + (1-\alpha) * \alpha * t_{n-1} + \dots + (1-\alpha)^j * \alpha * t_{n-j} + \dots + (1-\alpha)^{n+1} * \tau_0$$

23. Si illustri il significato e le conseguenze della cosiddetta anomalia di Belady (cap. 10)

L'anomalia di Belady si verifica nell'algoritmo di sostituzione delle pagine FIFO, essa consiste in un aumento inaspettato delle page fault all'aumentare dei frame assegnati ai processi, quando invece ci si aspetta che all'aumentare dei frame a disposizione dei processi, il numero di page fault generati diminuisca. Questa anomalia rende l'algoritmo FIFO inadatto e inaffidabile poiché è del tutto imprevedibile e quindi l'unica soluzione è optare per un altro algoritmo di sostituzione di pagina come ad esempio LRU che non presenta questa anomalia.

24. Chiarire le differenze (intrinseche e di gestione da parte del Sistema Operativo) tra gli interrupt sincroni e asincroni (cap. 2)

- Interrupt sincrono: processo che riceve un segnale di interrupt sincrono è costretto a sospendere immediatamente la sua esecuzione per cedere il controllo della CPU al SO che si occupa di avviare la gestione del determinato interrupt. Un interrupt sincrono non è quindi mascherabile e in genere è riferito ad errori di sistema come una divisione per 0 o un tentativo di accesso illegale alla memoria
- Interrupt asincrono: processo che riceve un segnale di interrupt asincrono può scegliere il momento più opportuno per sospendere la sua esecuzione (in un momento sicuro della sua esecuzione) per cedere il controllo della CPU al SO. Questo tipo di interrupt è mascherabile e in genere riguarda chiamate di sistema a servizi in modalità super utente

25. Si descrive brevemente la caratteristica di hold and wait collocandola nel contesto appropriato spiegando quali problematiche potrebbe creare e in quali casi. (cap. 8)

Hold and wait è una delle 4 condizioni che devono essere vere affinché si possa verificare un deadlock, consiste nel fatto che un processo che detiene delle risorse attende delle risorse in possesso da altri processi. Il contesto in cui si colloca è nel contesto del deadlock prevention, ossia si cerca di rendere non vera una delle 4 condizioni per far sì che non si possa verificare mai un deadlock.

Per rendere non vera la caratteristica di hold and wait bisogna far sì che un processo che richiede delle risorse non ne detenga altre, quindi un processo o chiede ed ottiene tutte le risorse di cui ha bisogno per completare la sua esecuzione oppure può richiedere delle risorse solo nel momento in cui non ne detiene altre.

Questo può portare ad uno spreco di risorse in quanto il loro utilizzo risulta poco efficiente, inoltre si può verificare la starvation, ossia processi che richiedono delle risorse molto usate e attendono indefinitamente

26. Si descrivano brevemente le caratteristiche degli algoritmi di scheduling mediante code multilivello. (cap.6)

Gli algoritmi di scheduling mediante code multilivello consistono nel dividere i processi in gruppi, ossia in code multiple, ad esempio processi in foreground e processi in background. Ogni coda ha un proprio algoritmo di scheduling, ad esempio la coda di foreground schedula con RR mentre la coda di background schedula con FCFS. Le code vengono schedulate, possibili algoritmi di scheduling per le code multilivello sono:

Scheduling preemptive a priorità fissa: ad esempio la coda di foreground ha priorità più alta rispetto alla coda dei processi background, possibile starvation. Partizionamento del tempo mediante time slice: esempio 80% tempo di CPU per la coda foreground e 20% CPU per la coda background.

27. Si esprima l'EAT in casi di presenza di TLB. Si chiarisca il contesto di utilizzo. (cap 9)

In ambito di memoria paginata, l'EAT è il tempo di accesso effettivo per la risoluzione di un indirizzo di memoria quando si fa uso della TLB, esso viene calcolato mediante l'uso di diversi parametri:

- Beta: tempo di accesso alla memoria
- Epsilon: tempo di accesso alla TLB (molto minore di beta)
- Alpha: Hit ratio, ossia percentuale di TLB hit

Il calcolo dell'EAT è di tipo probabilistico e fa uso del concetto di località spaziale e temporale:

$$EAT = \beta \cdot (2 - \alpha) + \epsilon$$

Conviene utilizzare la TLB quando l'EAT è minore del tempo di accesso alla memoria b moltiplicato per 2, che è il tempo di accesso totale alla memoria impiegato per la risoluzione di un indirizzo senza TLB in quanto sono richiesti due accessi alla memoria, uno per la Page Table e uno per l'indirizzo effettivo

28. Spiegare la struttura e descrivere le funzionalità dell'interrupt handler (cap.2)

L'interrupt handler è una routine di istruzioni per la gestione di un determinato interrupt. Quando la CPU riceve un segnale di interrupt effettua un polling per individuare da quale device proviene e che tipo di interrupt è, fatto questo si cerca all'interno del vettore di interrupt l'indirizzo alla routine di gestione dell'interrupt che si vuole gestire, questa routine è proprio l'interrupt handler.

29. Si descriva brevemente il fenomeno del convoy effect collocandolo nel contesto appropriato e spiegando in qual modo è possibile risolverlo (cap 6)

Il convoy effect è un fenomeno che si verifica nell'algoritmo di scheduling FCFS, esso consiste in più processi I/O Bound (Utilizzo breve della CPU, tanti I/O) attendono sistematicamente che un processo CPU Bound (lungo utilizzo della CPU, pochi I/O) termini di utilizzare la CPU. Si può risolvere in diversi modi:

- Algoritmo di scheduling a priorità con processi I/O Bound che hanno priorità maggiore dei processi CPU Bound, possibile starvation dei processi CPU Bound se c'è un flusso continuo di processi I/O Bound
- Algoritmo SJF (ottimale e di difficile implementazione)
- Algoritmo di schedulazione Round Robin assegna un quanto di tempo di CPU ad ogni processo gestendo la coda di ready come una lista circolare.

30. Si spieghi brevemente a cosa serve una Remote Procedure Call e qual è la funzione del client-stub all'interno di essa.

Una RPC è utilizzata nel momento in cui un processo client passa una procedure e i rispettivi parametri affinché sia eseguita da un processo server, il quale poi restituirà il risultato al client. Lo stub lato client si occupa del marshalling, ovvero della traduzione della procedure e dei parametri in formato External Data Representation, in modo che poi siano nuovamente tradotti dallo stub lato server in un formato adatto alla sua architettura.

31. Si descrivano le caratteristiche degli algoritmi di scheduling mediante cose multilivello (multilevel queue). Si chiarisca con precisione di quale meccanismo di schedulazione esse sono una approssimazione

32. Si descrivano le caratteristiche e la funzionalità dei sistemi a ipercubo

33. Qual è la condizione indispensabile perché sia possibile realizzare un semaforo?

un semaforo gestisce la sincronizzazione imponendo interattivamente un blocco attivo durante l'attesa (SPIN LOCK). lo spin lock viene utilizzato per impedire che più processi o più thread possano accedere simultaneamente ai dati condivisi e modificabili. Quindi p o th chiedono l'accesso ai dati condivisi protetti da uno spin lock che controlla se l'accesso è disponibile, se non lo è il processo o thread rimane in attesa. il semaforo è composto da:

S: variabile intera

- acquire(): metodo per accedere al semaforo
- relase(): metodo per rilasciare al semaforo

Per essere implementato necessita ovviamente che due thread o processi possano condividere memoria.

34. Si spieghi quale è il significato del permesso di esecuzione assegnato ad un file relativo ad un device.

Il permesso di esecuzione rispetto a file relativi a device non è significativo

35. Si tracci lo schema del contenuto di un Process Control Block evidenziando per ciascuna componente in quale circostanza della vita del processo viene creata/aggiornata.

il PCB è una struttura dati del kernel che contiene informazioni circa i processi quali:

- stato del processo
 - program counter
 - info sulla schedulazione della cpu
 - dispositivi I/O allocati al processo
 - info sulla gestione della memoria
 - quantità % di cpu utilizzata
 - registri della cpu
- Un processo viene creato ed inserito nella coda degli hold, aggiornato ad ogni context switching, e terminato quando finisce la propria esecuzione ed esce dalla memoria centrale.

36. Si spieghi brevemente a cosa serve una Remote Procedure Call e qual è la funzione del client-stub all'interno di essa.

Una RPC è utilizzata nel momento in cui un processo client passa una procedure e i rispettivi parametri affinché sia eseguita da un processo server, il quale poi restituirà il risultato al client. Lo stub lato client si occupa del marshalling, ovvero della traduzione della procedure e dei parametri in formato External Data Representation, in modo che poi siano nuovamente tradotti dallo stub lato server in un formato adatto alla sua architettura.

37. Si spieghi brevemente qual è la differenza (funzionale ed architetturale) tra lo short-term scheduler ed il long-term scheduler.

Lo schedulatore a lungo termine si occupa di inserire i processi pronti nella coda dei ready.

Lo schedulatore a breve termine si occupa di scegliere a quale processo allocare la cpu.

38. Quali sono le componenti del tempo di accesso a disco? Quale di esse tende ad essere dominante? È più conveniente effettuare pochi trasferimenti di blocchi di grosse dimensioni oppure molti trasferimenti di blocchi piccoli? Si motivino le risposte

Il tempo di accesso al disco è dato dal tempo di seek (tempo impiegato a raggiungere la traccia desiderata) e la latenza rotazionale (tempo impiegato a raggiungere il settore desiderato); nei dischi moderni quest'ultimo è sostanzialmente trascurabile per cui tende ad essere dominante il tempo di seek. È conveniente effettuare pochi trasferimenti in quanto si generano meno chiamate di I/O e si effettuano meno accessi al disco.